Numerical Python (NumPy)

Numerical Python (NumPy) is a powerful library in Python for numerical and matrix operations. It provides support for large, multi-dimensional arrays and matrices, along with mathematical functions to operate on these arrays efficiently.

Import/install necessary packages

```
import numpy as np
```

Array Creation

np.array(): Create an array from a list or tuple.

np.zeros(): Create an array filled with zeros.

np.ones(): Create an array filled with ones.

np.empty(): Create an array without initialising its values.

np.arange(): Create an array with regularly spaced values.

np.linspace(): Create an array with linearly spaced values.

np.eye(): Create a 2D identity matrix.

np.identity(): Create a square identity matrix.

Creating Arrays, Indexing, and Slicing Arrays

One-Dimensional and Two-Dimensional Arrays

A one-dimensional array, often referred to as a vector, is a fundamental data structure in NumPy. It represents a sequence of elements arranged along a single axis. A common example is a list of numbers.

A two-dimensional array, or matrix, extends the concept of a 1D array into two dimensions, forming rows and columns. It is a grid of elements where each element is uniquely identified by its row and column indices. Matrices are useful for representing tabular data and performing matrix operations.

```
empty_arr = np.empty(5)
print(empty_arr,'\n')

L1 = [1,2,3,4,5]
print(type(L1))
arr_1d = np.array(L1) # 1D Array : arr_1d = np.array([1, 2, 3, 4, 5])
print(type(arr_1d))
print(arr_1d)

print('\n')

L2 = [6,7,8,9,10]
arr_2d = np.array([L1, L2]) # 2D Array arr_2d = np.array([[1, 2, 3, 4, 5], [6,7,8,9,10]])
print(type(arr_2d))
print(arr_2d)
```

```
[0.   0.25 0.5  0.75 1.  ]

<class 'list'>
<class 'numpy.ndarray'>
[1 2 3 4 5]


<class 'numpy.ndarray'>
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]]
```

Common Methods

NumPy provides various methods for array creation, such as np.array().

Zeros and Ones

```
zeros_array = np.zeros((3, 4))   # 3x4 array of zeros
print (zeros_array)

print('\n')

ones_array = np.ones((5, 3))     # 5x3 array of ones
print(ones_array)
```

```
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]


[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
```

Filling:

```
filled_array = np.full((2, 3), 7)  # 2x3 array filled with 7
print(filled_array)
```

```
[[7 7 7]
 [7 7 7]]
```

Zeros and Ones like:

```
existing_array = np.array([[1, 2, 3], [4, 5, 6]])
print(existing_array,'\n')

zeros_like_array = np.zeros_like(existing_array)
print(zeros_like_array, '\n')

ones_like_array = np.ones_like(existing_array)
print(ones_like_array, '\n')
```

```
[[1 2 3]
 [4 5 6]]

[[0 0 0]
 [0 0 0]]

[[1 1 1]
 [1 1 1]]
```

Identity matrix:

```
identity_matrix = np.identity(4) # 4x4 identity matrix
print(identity_matrix,'\n')

identity_matrix_custom_size = np.eye(4)# 4x4 identity matrix
print(identity_matrix_custom_size,'\n')

identity_matrix_custom_size = np.eye(4,6)# 4x4 identity matrix with two additional columns with zeros
print(identity_matrix_custom_size,'\n')
```

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]

[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]

[[1. 0. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0.]
 [0. 0. 1. 0. 0. 0.]
 [0. 0. 0. 1. 0. 0.]]
```

Sequences:

```
sequence_1 = np.arange(10)
print("Sequences from 0 to 9:", sequence_1)

print('\n')

sequence_2 = np.arange(2, 11, 2)
print("A sequence of even numbers from 2 to 10:", sequence_2)
```

```
Sequences from 0 to 9: [0 1 2 3 4 5 6 7 8 9]


A sequence of even numbers from 2 to 10: [ 2  4  6  8 10]
```

Array Indexing, Slicing and Boolean Indexing

```
first_element=arr_1d[0] #Indexing
print(first_element)

Last_two_elements=arr_1d[1:4] #Slicing
print(Last_two_elements)

bool_arr=arr_1d>3 #Boolean Indexing
print(bool_arr)
```

```
1
[2 3 4]
[False False False  True  True]
```

Simple Array Operations:

```
print(arr_1d,'\n')
print(arr_2d,'\n')

sum_arr = arr_1d + arr_2d      # Element-wise addition (Basic Operation)
print(sum_arr,'\n')

sqrt_arr = np.sqrt(arr_1d)     # Element-wise square root (Universal Function)
print(sqrt_arr,'\n')

mean_value = np.mean(arr_2d)   # Calculating the mean of the entire array (Universal Function)
print(mean_value,'\n')
```

```
[1 2 3 4 5]

[[ 1  2  3  4  5]
 [ 6  7  8  9 10]]

[[ 2  4  6  8 10]
 [ 7  9 11 13 15]]

[1.         1.41421356 1.73205081 2.         2.23606798]

5.5
```

Broadcasting:

```
scaled_arr = arr_2d * 2    # Broadcasting scalar to a 2D array
print(scaled_arr,'\n')
```

```
[[ 2  4  6  8 10]
 [12 14 16 18 20]]
```

Array Manipulation

np.reshape(): Reshape an array.

np.flatten(): Flatten a multi-dimensional array to a 1D array.

np.transpose(): Transpose an array.

np.concatenate(): Concatenate arrays along a specified axis.

```
reshaped_arr = arr_1d.reshape((5, 1))   # Reshaping to a column vector
print(reshaped_arr ,'\n')

concat_arr = np.concatenate((arr_1d, arr_2d.flatten()))   # Concatenating arrays and flattening an array
print(concat_arr,'\n')

transpose_arr=np.transpose(arr_2d) # Transposing an array
print(transpose_arr,'\n')
```

```
[[1]
 [2]
 [3]
 [4]
 [5]]

[ 1  2  3  4  5  1  2  3  4  5  6  7  8  9 10]

[[ 1  6]
 [ 2  7]
 [ 3  8]
 [ 4  9]
 [ 5 10]]
```

Random Number Generation

np.random.rand(): Generate random numbers from a uniform distribution (0 to 1).

np.random.randn(): Generate random numbers from a standard normal distribution.

np.random.randint(): Generate random integers within a specified range.

```
random_number_arr_1d = np.random.rand(5) # 1D array with 5 random numbers from a uniform distribution [0, 1)
print(random_number_arr_1d,'\n')

random_number_arr_2d = np.random.rand(5,4) # 5x4 array of random numbers from a uniform distribution [0, 1)
print(random_number_arr_2d,'\n')

random_values_normal = np.random.randn(4, 4) # 4x4 array of random numbers from a standard normal distribution
print(random_values_normal,'\n')

random_number_arr_2d = np.random.randint(4,10, size=(5, 4)) # 5x4 array of random integer numbers from a uniform distributic
print(random_number_arr_2d,'\n')
```

```
[0.06919435 0.26108056 0.09861436 0.43406042 0.92721005]

[[0.78233824 0.84142667 0.81528437 0.41793372]
 [0.00179475 0.70156598 0.04422399 0.46610594]
 [0.98986724 0.5389722  0.6843519  0.97843288]
 [0.03047481 0.72147092 0.96172422 0.53528981]
 [0.88596527 0.31202217 0.62355657 0.98955068]]

[[-1.91992303 -2.29928708  0.72584029 -1.86580334]
 [ 0.34089459  0.93924501 -0.84623492 -0.12536279]
 [-0.82009772  0.13104683 -0.21476891  1.19522214]
 [-0.6431213   2.26018645 -1.46526121 -0.90519898]]

[[5 9 7 5]
 [7 5 4 6]
 [4 7 4 7]
 [6 5 5 5]
 [8 4 8 6]]
```

Mathematical Operations

np.add(): Add the corresponding elements of two arrays.

np.subtract(): Subtract the corresponding elements of two arrays.

np.multiply(): Multiply the corresponding elements of two arrays.

np.divide(): Divide the corresponding elements of two arrays.

np.power(): Raise elements to a specified power.

np.sqrt(): Compute the square root of each element.

```
print(np.add(arr_1d,arr_1d),'\n')
print(np.subtract(arr_1d,arr_1d),'\n')
print(np.multiply(arr_1d,arr_1d),'\n')
print(np.multiply(arr_1d,arr_2d),'\n')
print(np.divide(arr_1d,arr_2d),'\n')
print(np.power(arr_1d,2),'\n')
print(np.sqrt(arr_1d),'\n')
```

```
[ 2  4  6  8 10]

[0 0 0 0 0]

[ 1  4  9 16 25]

[[ 1  4  9 16 25]
 [ 6 14 24 36 50]]

[[1.         1.         1.         1.         1.        ]
 [0.16666667 0.28571429 0.375      0.44444444 0.5       ]]

[ 1  4  9 16 25]

[1.         1.41421356 1.73205081 2.         2.23606798]
```

Statistical Functions

np.mean(): Compute the mean of an array.

np.median(): Compute the median of an array.

np.std(): Compute the standard deviation of an array.

np.sum(): Compute the sum of array elements.

```
print(np.sum(arr_1d),'\n')
print(np.mean(arr_1d),'\n')
print(np.std(arr_1d),'\n')
print(np.median(arr_1d),'\n')
```

```
15

3.0

1.4142135623730951

3.0
```

Linear Algebra

np.dot(): Compute dot product of two arrays.

np.linalg.inv(): Compute the inverse of a matrix.

Note: For 1D arrays, the dot product is equivalent to the sum of the element-wise product. For 2D arrays, matrix multiplication rules apply.

```
print(np.dot(4,5),'\n')
print(np.dot(arr_1d,arr_1d),'\n')
print(np.dot(arr_2d,np.transpose(arr_2d)),'\n')
print(np.linalg.inv(np.eye(3)),'\n')
```

```
20

55

[[ 55 130]
 [130 330]]

[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Logical Operations

np.logical_and(), np.logical_or(), np.logical_not(): Perform element-wise logical operations.

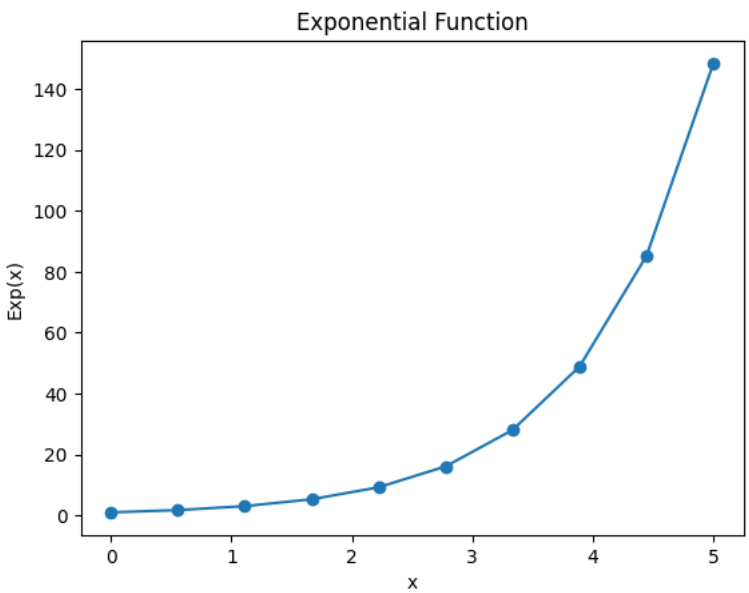
```
A1 = np.array([True, True, False, False])
A2 = np.array([True, False, True, False])

logical_and_result = np.logical_and(A1,A2)
print("Logical AND Result:")
print(logical_and_result)


logical_or_result = np.logical_or(A1,A2)
print("\nLogical OR Result:")
print(logical_or_result)


logical_not_result = np.logical_not(A1)
print("\nLogical NOT Result:")
print(logical_not_result)
```

```
Logical AND Result:
[ True False False False]

Logical OR Result:
[ True  True  True False]

Logical NOT Result:
[False False  True  True]
```

Extras:

```
import matplotlib.pyplot as plt  # Importing matplotlib for visualisation
linear_values = np.linspace(0, 5, 10) # An array with 10 linearly spaced values between 0 and 1
print(linear_values,'\n') # Print for your understanding
plt.plot(linear_values, np.exp(linear_values), marker='o')
plt.title('Exponential Function')
plt.xlabel('x')
plt.ylabel('Exp(x)')
plt.show()
```

```
[0.         0.55555556 1.11111111 1.66666667 2.22222222 2.77777778
 3.33333333 3.88888889 4.44444444 5.        ]
```



Exercise 1: Revenue Analysis using NumPy

In a retail business, suppose you have data on the number of units sold and the unit price for different products over a certain period.

1. Use NumPy to compute the total revenue for each product and
2. the overall revenue for the business.

Data:

units_sold: 50, 30, 40, 25, 60

unit_price: 20.0, 15.5, 18.0, 22.5, 12.0

Tips: np.array(), np.sum(), np.mutltiply()

```
# import numpy
# use sample data: Number of units sold and unit price for each product
# 1. Compute the total revenue for each product
# 2. Compute the overall revenue for the business
```

Exercise 2: Revenue Forecasting in a Retail Business using NumPy

Let us consider a simple scenario where a retail business wants to forecast its monthly revenue based on historical data.

Data:

historical_revenue ('000): 120, 150, 130, 110, 160, 140, 170, 180, 200, 190, 220, 210

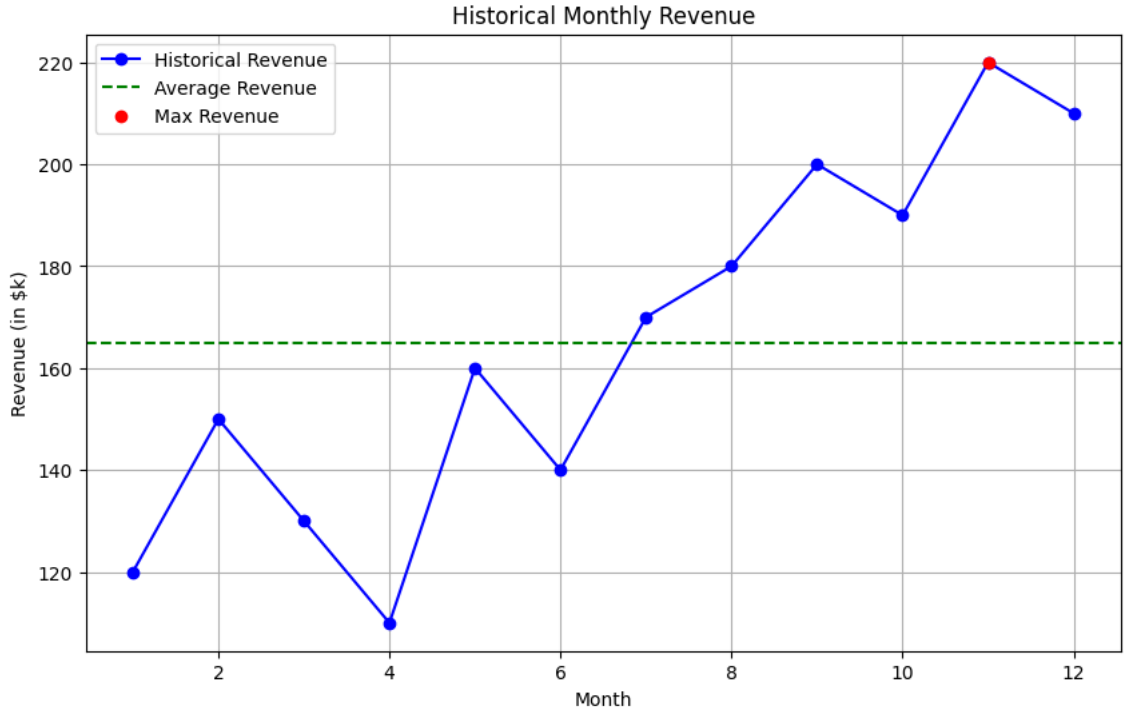
The Code performs the following tasks:

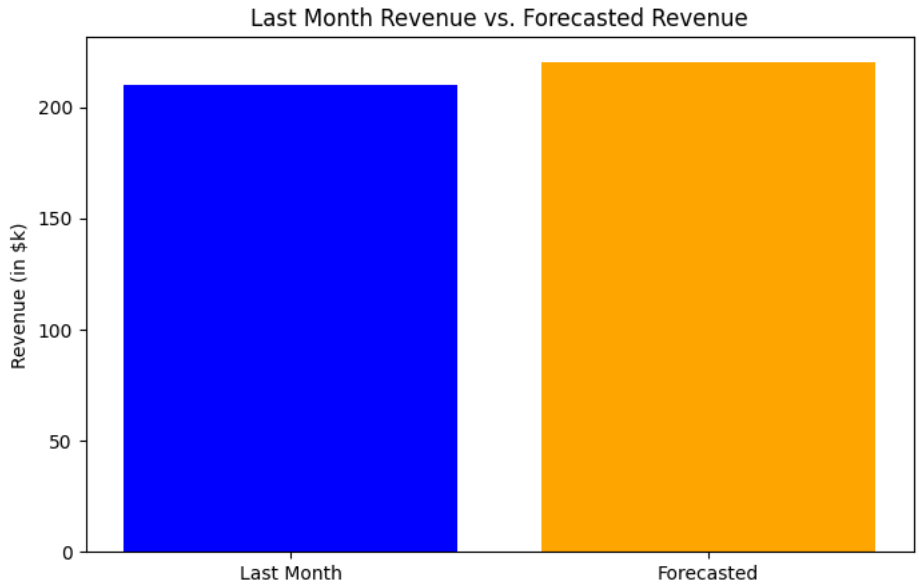
1. Calculates the average monthly revenue.
2. Identifies the month with the highest revenue.
3. Calculates the percentage growth from the first to the last month.
4. Forecasts next month's revenue assuming a 5% growth rate.
5. Plots the historical revenue data.

Tasks:

1. Obtain the Lowest Revenue: Expand the code to identify and calculate the lowest monthly revenue, provide a comprehensive view of the range of revenue fluctuations.
2. Enhanced Historical Revenue Plot: Modify the historical revenue plot to incorporate a yellow dot representing the month with the lowest revenue. Replace the mean line with a red median line, offering an alternative measure of central tendency. Additionally, recalculate and plot the forecasted revenue for the next month, this time assuming a 10% growth rate.

```
import numpy as np
import matplotlib.pyplot as plt

historical_revenue = np.array([120, 150, 130, 110, 160, 140, 170, 180, 200, 190, 220, 210])

# The average monthly revenue
average_revenue = np.mean(historical_revenue)

# The month with the highest revenue
max_revenue_month = np.argmax(historical_revenue) + 1  # Adding 1 to match with month index
max_revenue = historical_revenue[max_revenue_month - 1]

# The percentage growth from the first to the last month
initial_monthly_revenue = historical_revenue[0]
final_monthly_revenue = historical_revenue[-1]
percentage_growth = ((final_monthly_revenue - initial_monthly_revenue) / initial_monthly_revenue) * 100

# Forecast next month's revenue assuming a 5% growth rate
forecast_growth_rate = 0.05
forecasted_revenue = final_monthly_revenue * (1 + forecast_growth_rate)

# Plot
months = np.arange(1, 13)
plt.figure(figsize=(10, 6))
plt.plot(months, historical_revenue, marker='o', label='Historical Revenue', color='blue')
plt.axhline(y=average_revenue, linestyle='--', label='Average Revenue', color='green')
plt.scatter(max_revenue_month, max_revenue, color='red', label='Max Revenue', zorder=5)
plt.title('Historical Monthly Revenue')
plt.xlabel('Month')
plt.ylabel('Revenue (in $k)')
plt.legend()
plt.grid(True)
plt.show()

# Plot: the forecasted revenue
plt.figure(figsize=(8, 5))
plt.bar(['Last Month', 'Forecasted'], [final_monthly_revenue, forecasted_revenue], color=['blue', 'orange'])
plt.title('Last Month Revenue vs. Forecasted Revenue')
plt.ylabel('Revenue (in $k)')
plt.show()
```





Extras:

Plot the 13th month forecast in the same line chart.

Array Creation:

np.array(): Create an array from a list or tuple.

np.zeros(): Create an array filled with zeros.

np.ones(): Create an array filled with ones.

np.empty(): Create an array without initialising its values.

np.arange(): Create an array with regularly spaced values.

np.linspace(): Create an array with linearly spaced values.

np.eye(): Create a 2D identity matrix.

np.identity(): Create a square identity matrix.

Array Manipulation:

np.reshape(): Reshape an array.

np.flatten(): Flatten a multi-dimensional array to a 1D array.

np.transpose(): Transpose an array.

np.concatenate(): Concatenate arrays along a specified axis.

Mathematical Operations:

np.add(): Add the corresponding elements of two arrays.

np.subtract(): Subtract the corresponding elements of two arrays.

np.multiply(): Multiply the corresponding elements of two arrays.

np.divide(): Divide the corresponding elements of two arrays.

np.power(): Raise elements to a specified power.

np.sqrt(): Compute the square root of each element.

Statistical Functions:

np.mean(): Compute the mean of an array.

np.median(): Compute the median of an array.

np.std(): Compute the standard deviation of an array.

np.sum(): Compute the sum of array elements.

Random Number Generation:

np.random.rand(): Generate random numbers from a uniform distribution (0 to 1).

np.random.randn(): Generate random numbers from a standard normal distribution.

np.random.randint(): Generate random integers within a specified range.

Linear Algebra: